



探索 Kubernetes 管理 OceanBase 数据库的优势与 策略

分享人: 孙宏鑫 (雪染)





Content 目录

01 背景介绍

02 典型做法

03 我们的实践

04 Demo



Part 01

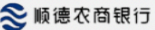
背景介绍

OceanBase 简介

1.0 时代: 坚定走向分布式架构

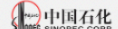
2010	2013	2014
产品立项	扩大使用范围	核心交易上线
第一个用户	多个业务系统	支付宝交易 金融级核心业务
	 	

2.0 时代: 原生分布式数据库

2016	2017	2019
全业务覆盖	多家金融客户	打破世界纪录
核心账务 核心交易支付	互联网 核心系统	Oracle 兼容 公有云服务 TPC-C 6088万
    	 四川省农村信用社  顺德农商银行  南京银行	  招商证券 广东农信   常熟农商银行 中国人民保险   五洲银行 西安银行   天津银行 浙商证券

3.0 时代: 混合引擎、混合部署

4.0 时代: 分布式一体化架构

2020	2021	2022
独立商业化	规模化推广	公有云走向海外
HTAP 引擎 TPC-C 7.07亿 走向通用行业、更 多头部客户核心系 统	TPC-H 1526万 社区版发布 试点海外客户	单机分布式一体化架构 公有云上线 公有云北美站点开服
 中国移动  中国工商银行  中国石化  中华保险	 江西省人力资源和社会保障厅  交通银行  DANA  GCash  上投摩根  数字江西	 海底捞  理想  致欧  TELECOM ITALIA  ANT BANK 蚂蚁银行  easypaisa

分布式 KV 存储

SQL 引擎，多副本高可用

兼容性，透明扩展，多活容灾

批量处理，企业级特性，HTAP

OceanBase 简介

自研一体化架构兼容经典模式，实现单机和分布式、TP和AP的融合

—原生多租户架构—

Oracle 兼容

MySQL 兼容

— HTAP 引擎 (TP + AP) —

SQL 优化引擎

并行执行引擎

存储过程

—集中式 / 分布式一体化架构—

分布式存储

分布式事务

分布式调度

—基于 Paxos+ 数据同步的灵活的容灾架构—

单机房三副本

同城三机房

双机房主备

两地三中心

三地五中心

—灵活的部署模式—

物理部署

专有云/混合云

公有云/多云

完全自主研发的数据库，能做到核心替代的真正落地

★ 原生分布式

全量数据校验真正实现数据强一致，数据不丢失，高可用，平滑扩展

★ 单机分布式一体化

自研一体化架构突破高性能和高可用，实现应用无限扩展和服务永远在线

★ HTAP

一份数据既能做事务处理又能实时分析，通过 HTAP 助力拓展更多可能

★ MySQL平滑迁移

业务少量修改甚至不改即可迁移到OB，自动评估和迁移工具

★ 低成本

基于 LSM-Tree 的高压缩引擎平衡了“性能”和“压缩”的瓶颈，有效降低存储成本 70% - 90%

★ 多租户

资源隔离按需使用，灵活管理，适合微服务架构和 SaaS 行业应用

OceanBase 简介

OceanBase 是全球唯一在事务处理和数据分析，两个领域测试中都获得过世界第一的中国自研数据库

2020 年 5 月 25 日

OceanBase 在事务型基准测试 TPC-C 中

7.07亿

tpmC 创下新的世界纪录

2021 年 5 月 19 日

OceanBase 在分析型基准测试 TPC-H 中

1526万

QphH@30000GB 的性能总分创下新的世界纪录

OceanBase 简介

以下数据来自于**实际**生产系统

6100

万次/秒

数据库峰值处理能力

> 1000

台

单集群节点数

> 6

PB

单库存储容量

> 3200

亿行

单表行数

RPO=0,
RTO<8

秒

少数副本故障时

OceanBase 简介

已助力1000+ 客户实现关键业务系统升级

从金融走向非金融，从中国走向全球



使用 Kubernetes 管理数据库的需求

容器化的趋势

1. 标准化: 容器化提供了标准的环境, 解决了环境依赖的问题
2. 隔离性: Namespace 隔离, 不同namespace下的进程不可见, 资源隔离, 限制容器能够使用的资源
3. 资源弹性: 资源池化, 可以动态的扩缩容, 结合业务的峰谷, 提升资源使用效率
4. 可编排: 结合 kubernetes 技术, 可以实现复杂的应用部署形态

数据库的发展

1. 分布式: 解决单机数据库容量和性能的问题, 使用水平扩展带来无限的计算能力
2. 与云设施深度结合: 通过与云基础设施的结合, 能够更快速与可靠的部署数据库服务
3. AI自治: 结合 AI 技术实现数据库的自监控、自调优、自诊断、自运维

数据库管理方式的对比

自己管理

可用性: 取决于基础设施和部署方案

成本: 需要包含基础设施的成本, 规模较小时相对来说投入比较大

运维: 依靠运维人员的能力或者自己开发工具

云服务

可用性: 云厂商提供解决方案, 一般可以保证较高的可用性

成本: 前期成本比较少, 按量付费, 扩容方便

运维: 运维负担较小, 但是相对来说自己掌控程度不高

Kubernetes

可用性: K8s 提供了许多 feature 可以利用

成本: 资源动态扩展, 与云服务的方式相当

运维: 减轻了手动运维的复杂性, 保留了可控性



Part 02

使用 Kubernetes 部署数据库应用的典型做法

Kubernetes 部署数据库的挑战

运维复杂性

数据库作为典型的有状态应用并不能像无状态应用一样随意起停，往往需要考虑节点的运行顺序，亲和性等，这些对于不同的数据库来说是不一样的

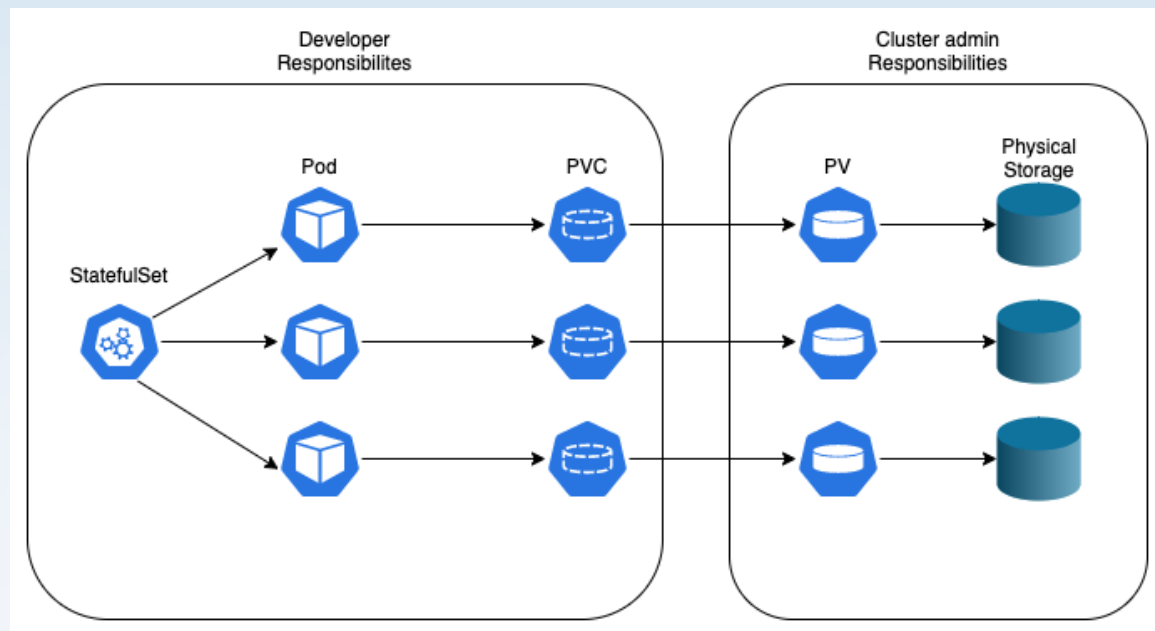
数据存储

数据库作为存储系统，其最重要的部分就是存储的数据，而容器的重启和升级是比较频繁发生的事情，为了保证数据的可用性，往往需要分开管理存储系统和 pod

网络

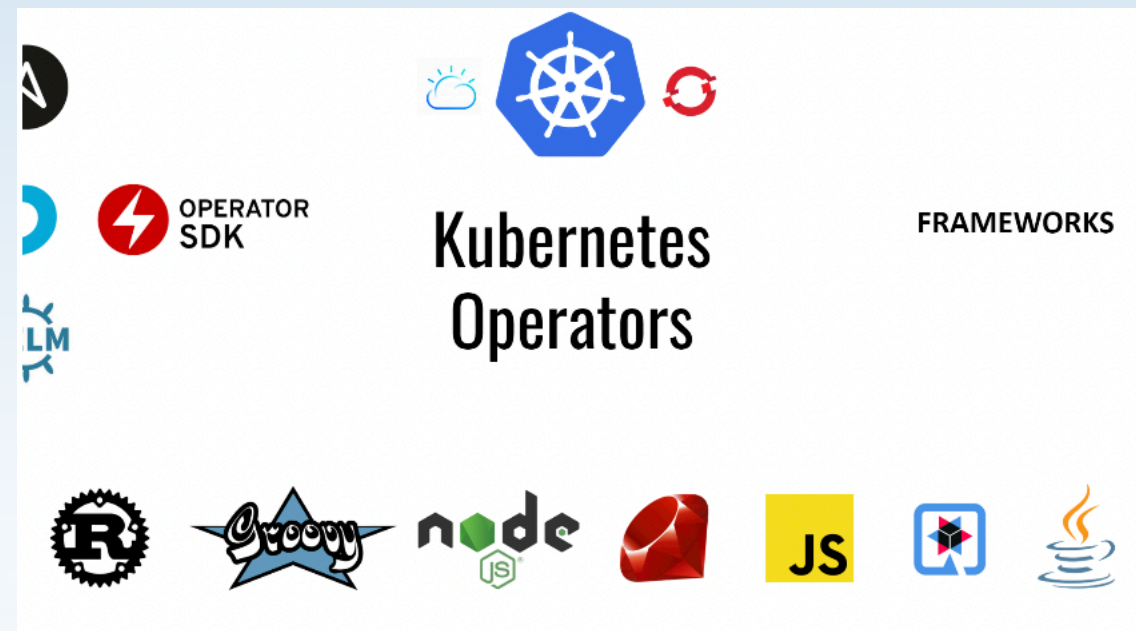
默认的网络地址是随机分配的，但是由于数据库的一些特性，往往对于网络有一定的要求，这需要额外的做出一些努力来实现

Kubernetes 部署数据库的模式



StatefulSet模式

适用于有状态的数据库，可提供稳定的网络标识和持久化存储。

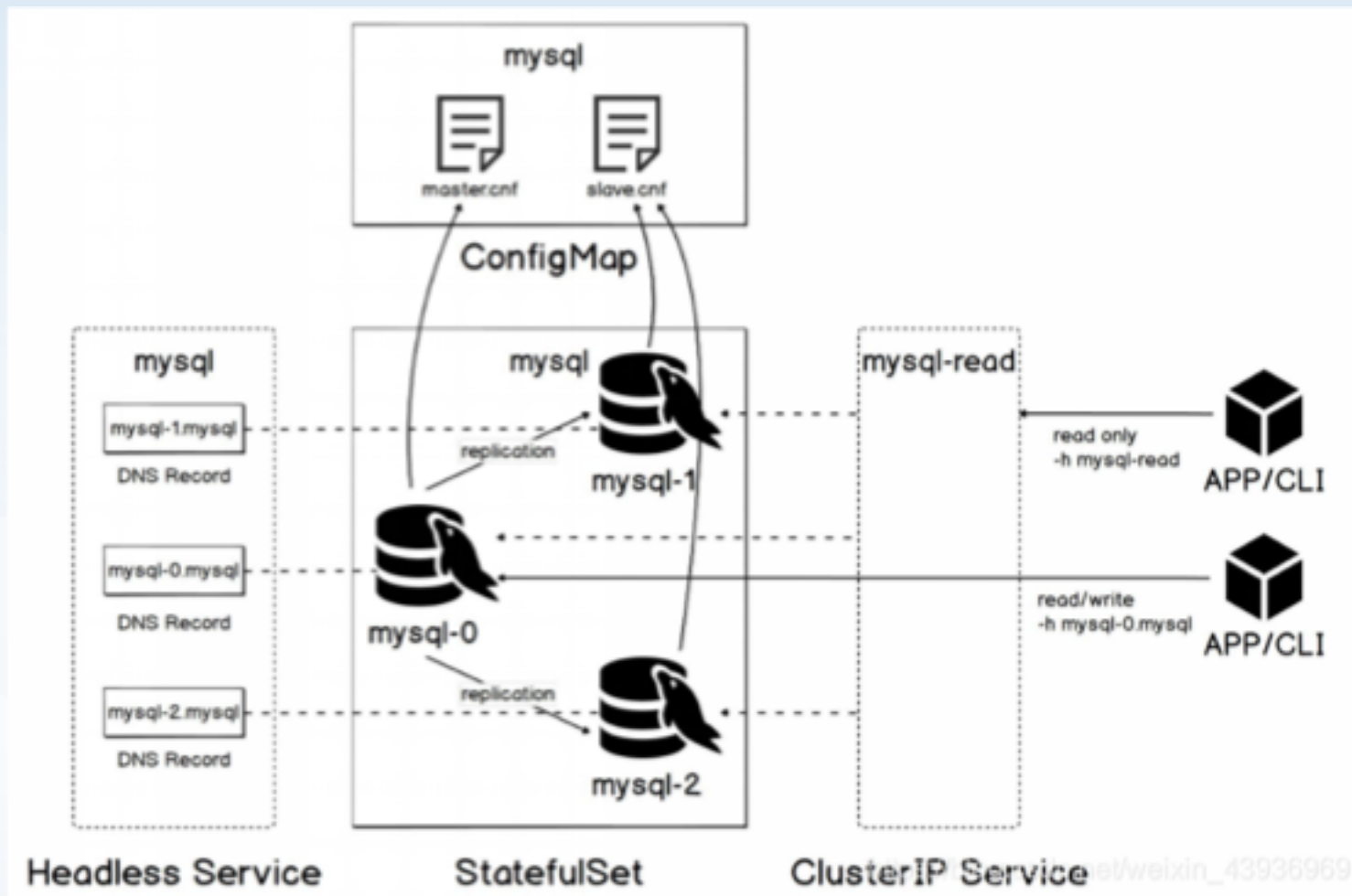


Operator模式

自定义的 operator 可以简化数据库的部署、配置和管理，提供更高級的运维能力。

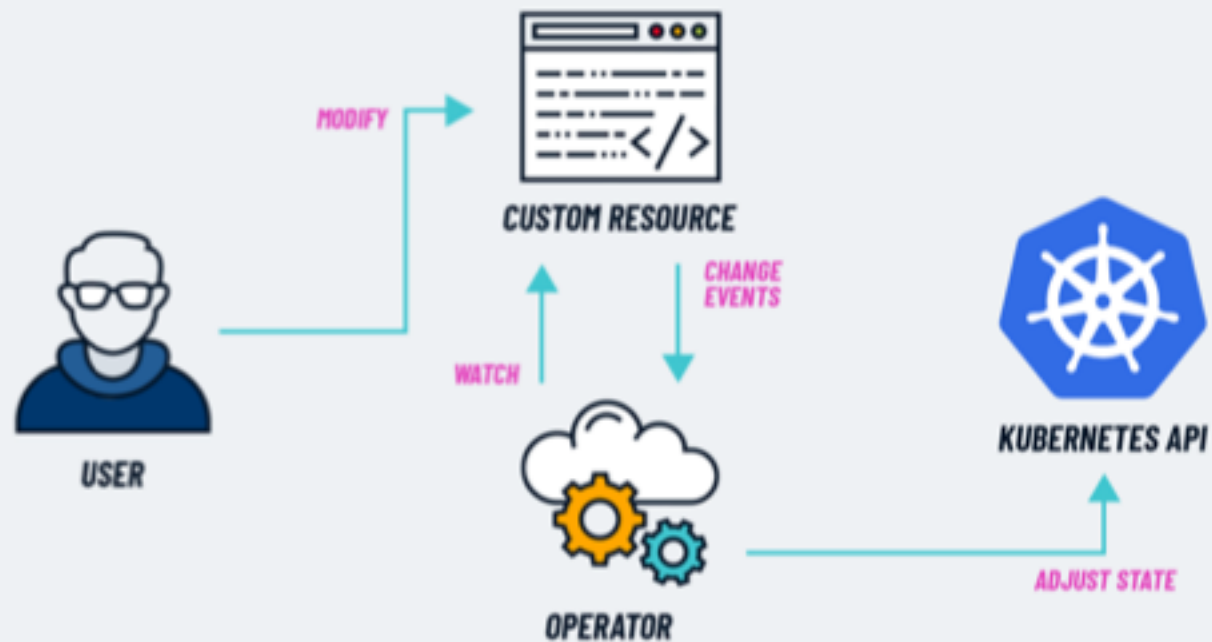
StatefulSet 模式部署数据库

- 唯一网络 ID
- 部署顺序
- 升级顺序



Operator 模式部署数据库

- 自定义资源
- 声明式配置
- 支持复杂的运维逻辑



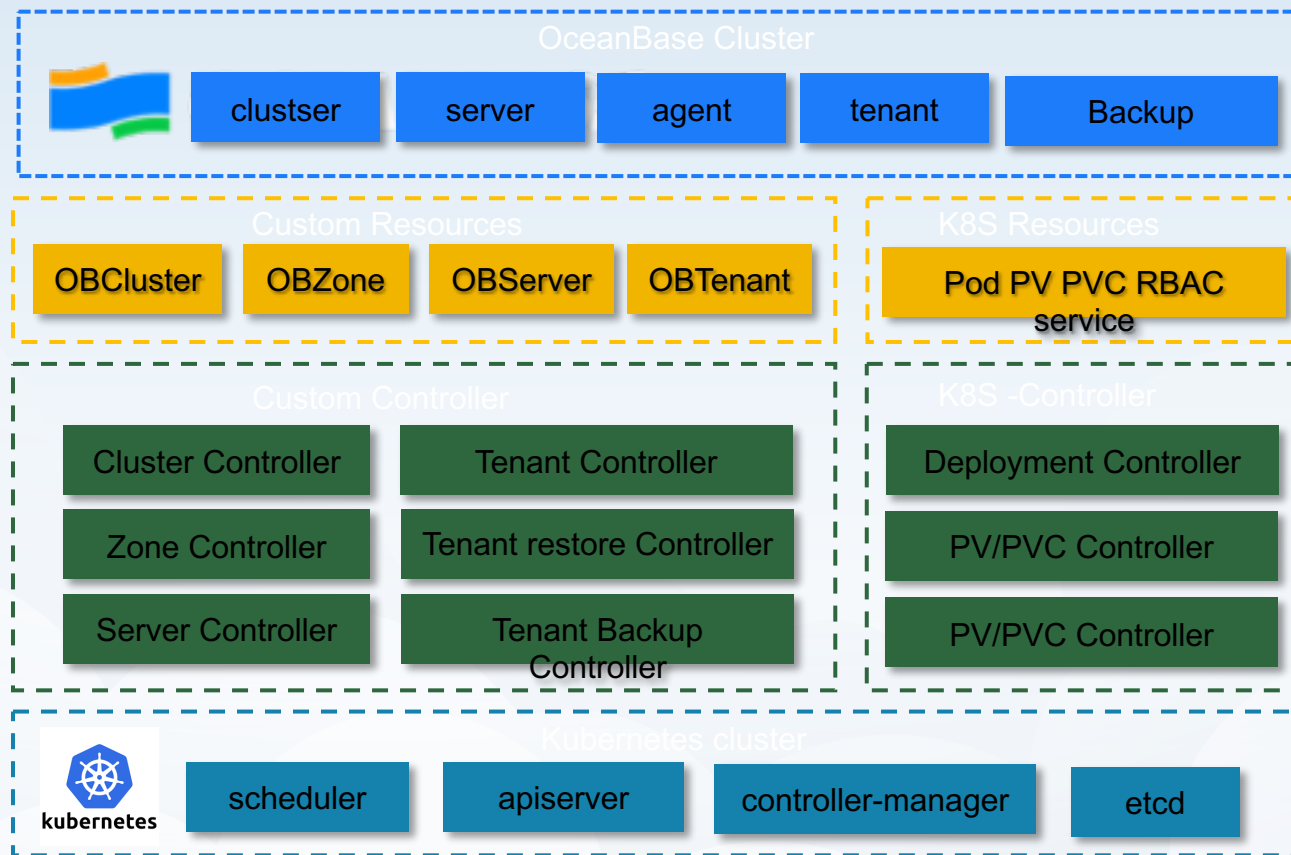


Part 03

如何使用 Kubernetes 管理 OceanBase 数据库

整体架构

1. 定义OceanBase以及相关组件的资源
2. 实现资源的控制器，操作K8s相关的资源
3. 像使用 K8s 其他类型资源一样来声明式的使用 OceanBase



资源定义

obcluster

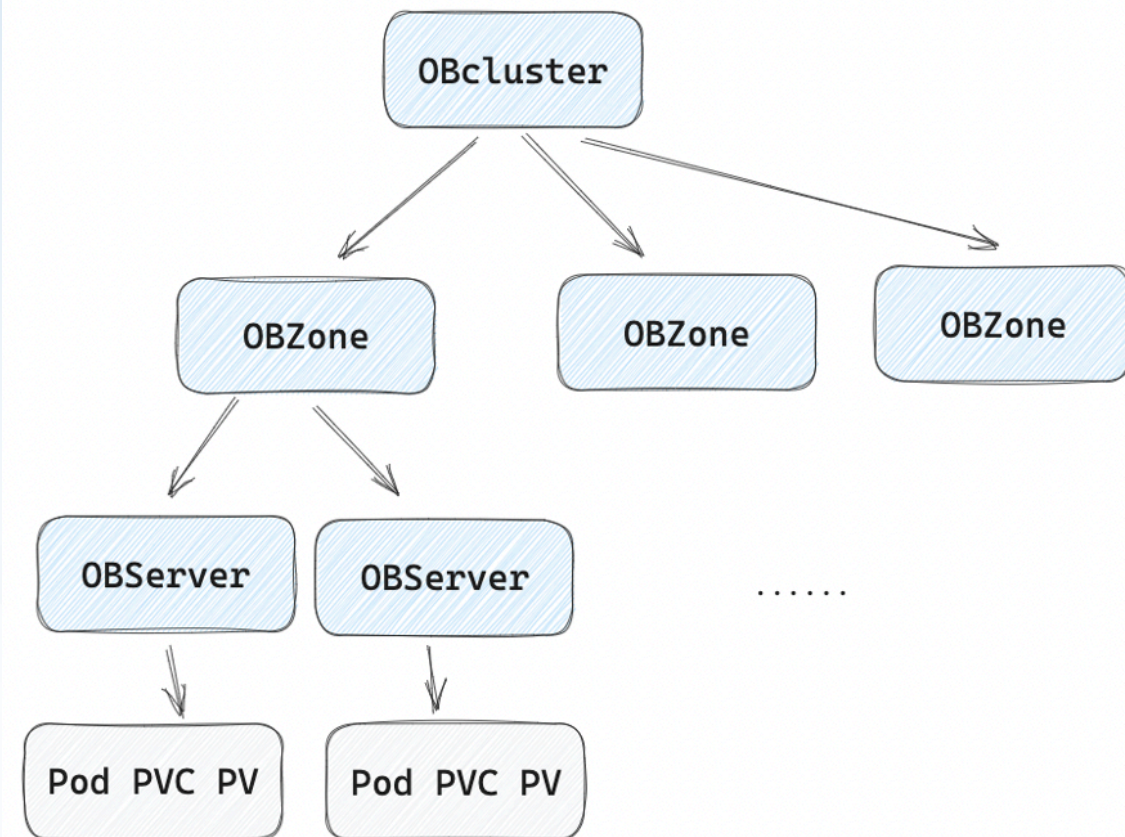
OceanBase 集群资源的定义, 描述了整个 OceanBase 集群的定义和状态

obzone

OceanBase zone 资源的定义, 描述了 OceanBase zone 的定义和状态

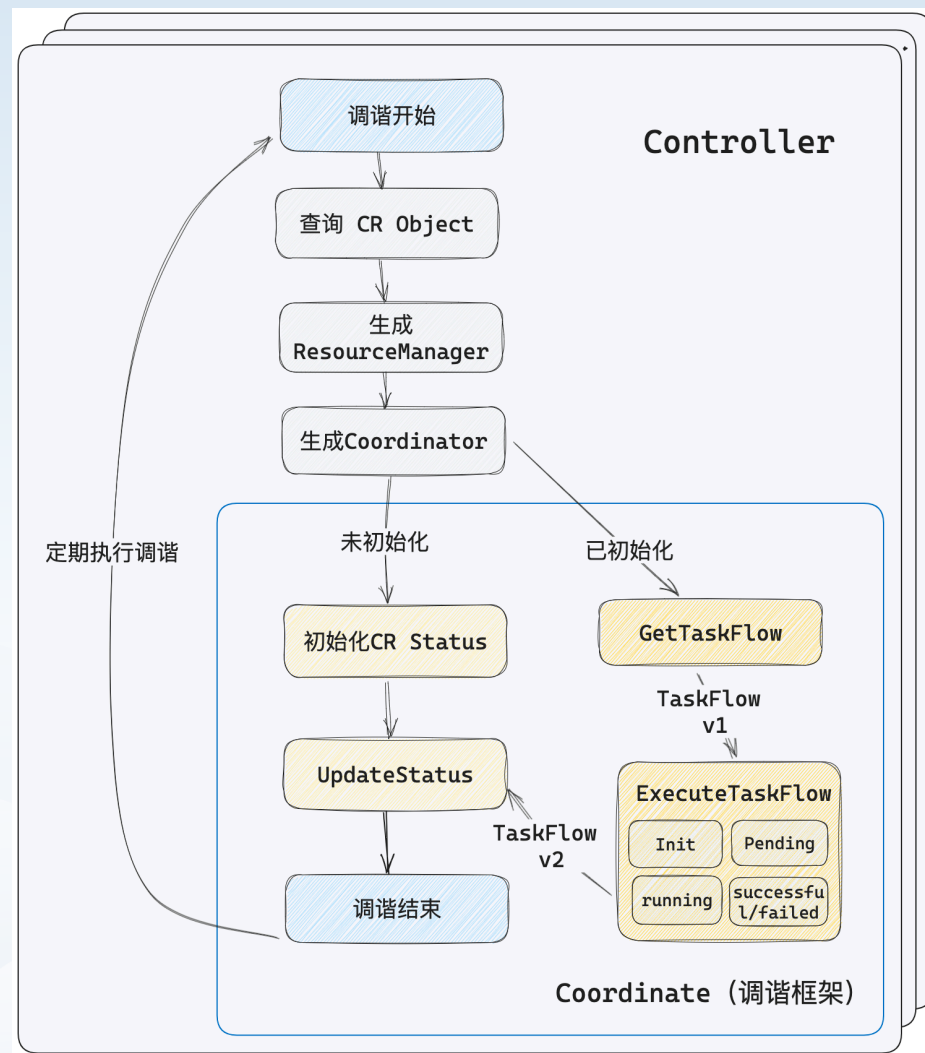
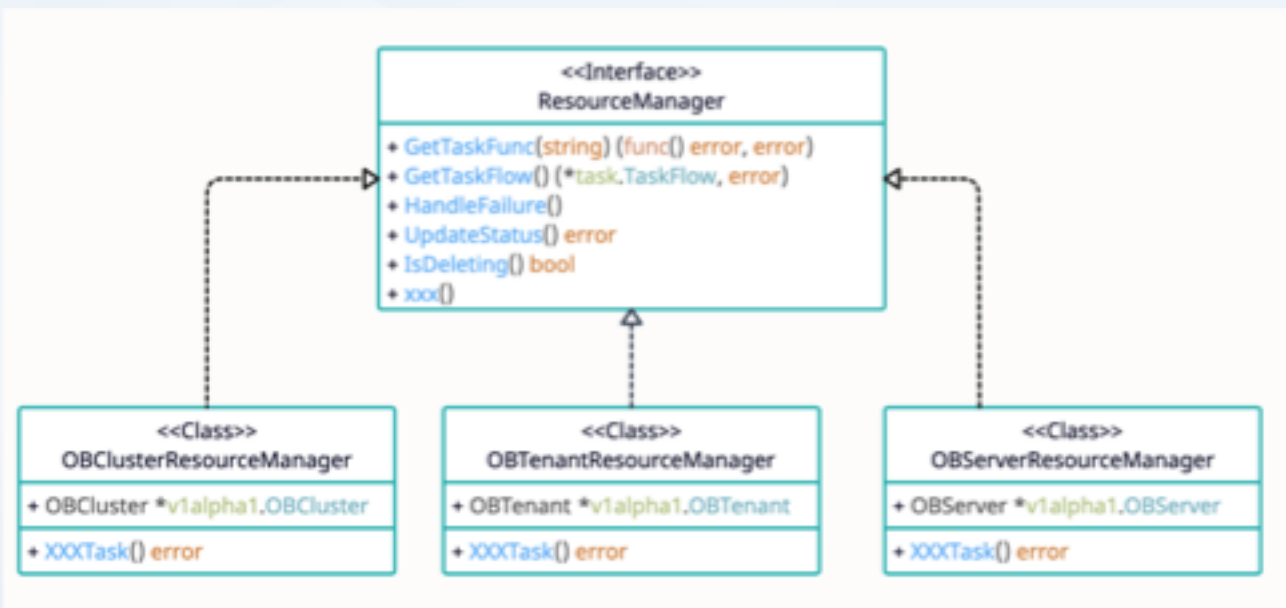
observer

OceanBase server 资源的定义, 描述了 OceanBase server 的定义和状态

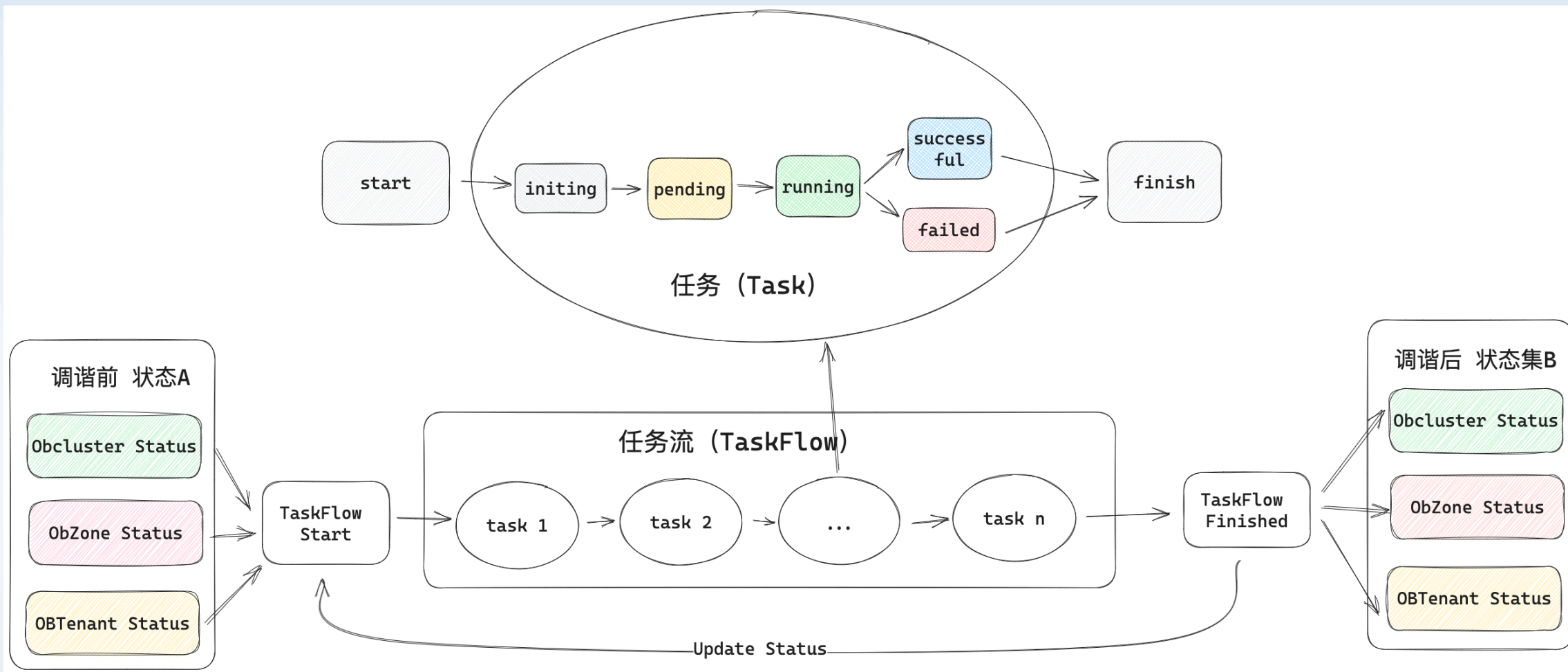


调谐流程

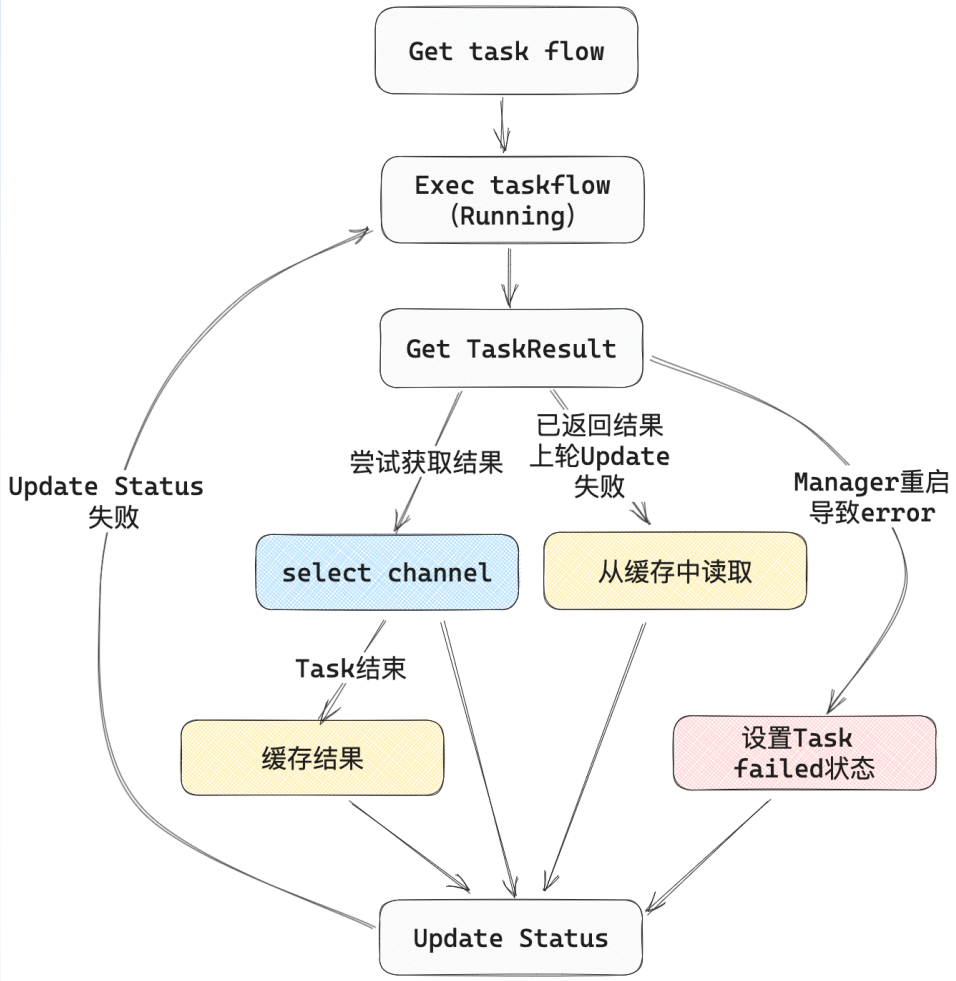
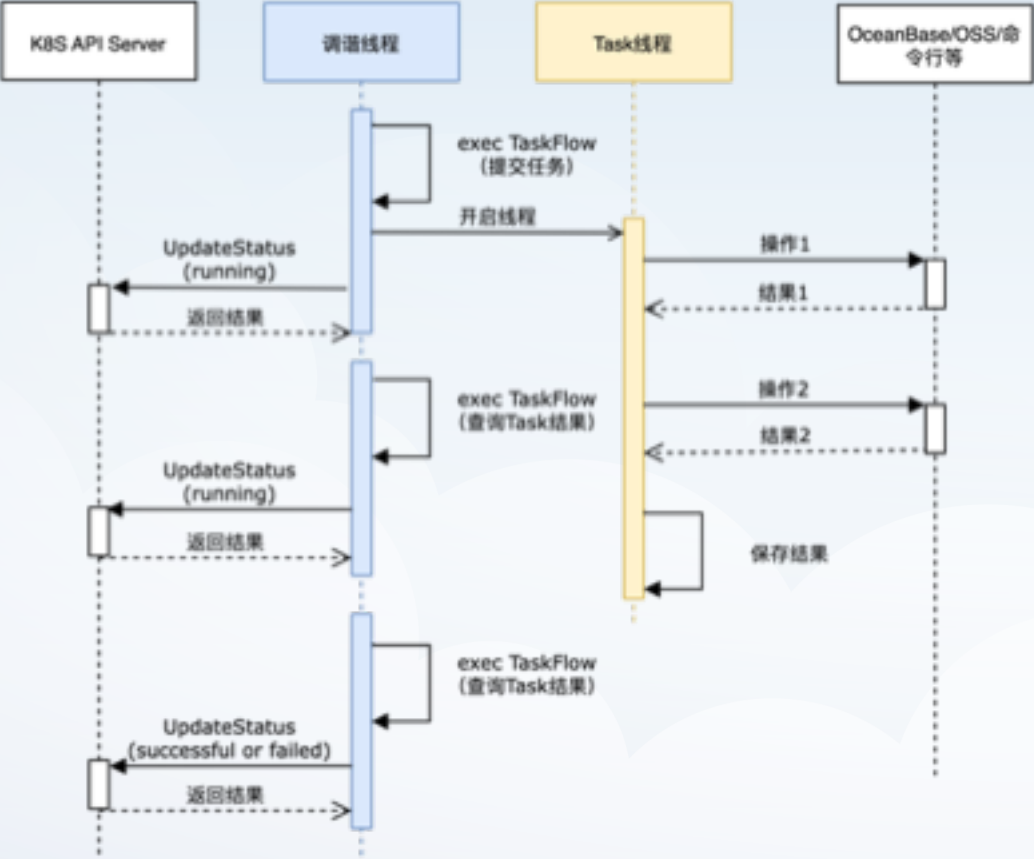
- Coordinator: 定义统一的资源调谐流程
- ResourceManager: 每个资源调谐相关接口
- TaskFlow: 任务流，编排一些列运维操作。



异步任务实现



异步任务执行



任务异常处理

1. Start over: 从头开始执行，默认策略
2. Retry current: 重试当前任务
3. Pause: 暂停, 一般用于非常危险的操作



Kubernetes 部署 OceanBase 的挑战

运维复杂性

1. 通过 ob-operator 实现对于 OceanBase 相关资源的定义，自定义运维逻辑
2. 标准化的资源调谐流程，根据有限的状态来生成任务流

数据存储

1. 存储与 Pod 的生命周期分离，得益于 K8s 的存储方案，相比于物理机有更多的选择性
2. 得益于 OceanBase 的架构设计，本身具备高可用的特性，对存储的依赖相对较少

网络

1. Pod IP 地址变化，使用了 CNI 的特性，增加保持 IP 地址的处理逻辑
2. OceanBase 可以容忍少数派的 Pod 发生变化，当作新的 observer 进行动态扩容

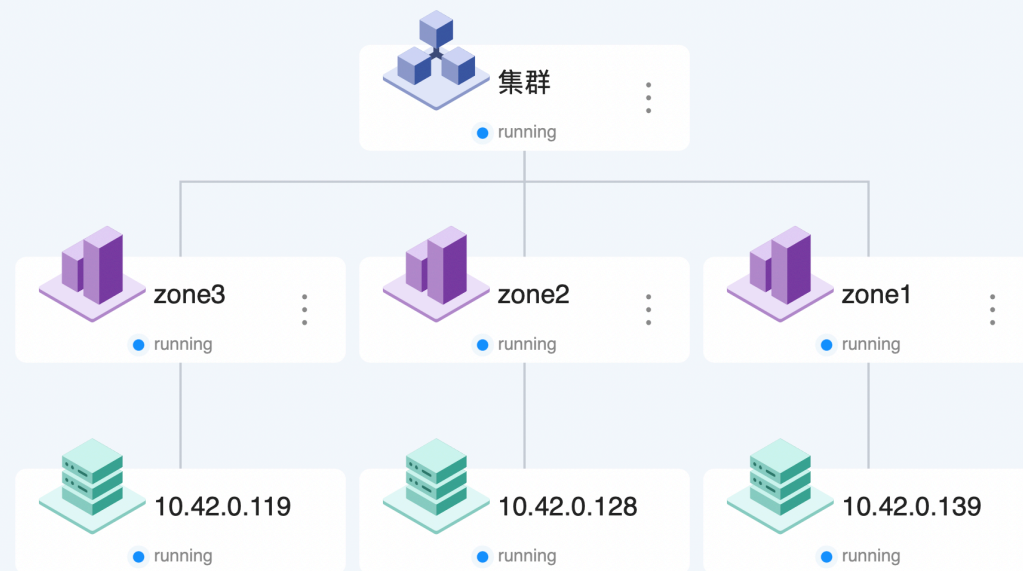


Part 04

Demo

部署一个集群

```
apiVersion: oceanbase.oceanbase.com/v1alpha1
kind: OBCluster
metadata:
  name: test
  namespace: oceanbase
spec:
  clusterName: obcluster
  clusterId: 1
  userSecrets:
    root: root-password
    proxyro: proxyro-password
    monitor: monitor-password
    operator: operator-password
  topology:
    - zone: zone1
      replica: 1
      # nodeSelector:
      #   k1: v1
      # affinity:
      #   nodeAffinity:
      #     podAffinity:
      #     podAntiAffinity:
      #   tolerations:
      #     - key: "obtopo"
      #       value: "zone"
      #       effect: "NoSchedule"
    - zone: zone2
      replica: 1
    - zone: zone3
      replica: 1
  observer:
    image: oceanbase/oceanbase-cloud-native:4.2.1.1-101010012023111012
    resource:
      cpu: 2
      memory: 10Gi
    storage:
      dataStorage:
        storageClass: local-path
        size: 50Gi
      redoLogStorage:
        storageClass: local-path
        size: 50Gi
      logStorage:
        storageClass: local-path
        size: 20Gi
  monitor:
    image: oceanbase/obagent:4.2.1-100000092023101717
    resource:
      cpu: 1
      memory: 1Gi
```



kubectl apply -f obcluster.yaml

未来规划

更可靠

1. 结合内核的能力，实现更好的故障恢复方案，不依赖特定的组件即可实现可靠的容灾恢复策略
2. 支持更丰富的容灾部署方案，比如部署在多个 K8s 集群之间

更易用

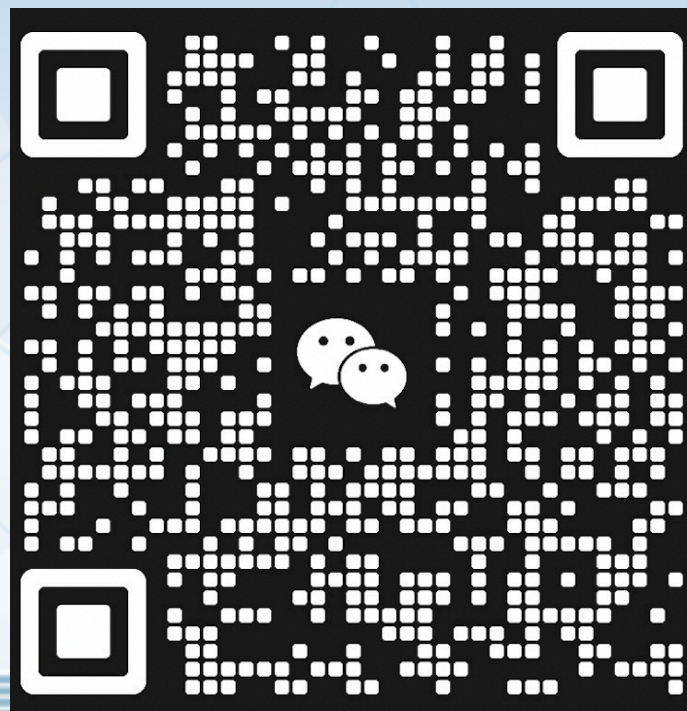
1. 提供友好的 dashboard，可以用来管理 K8s 中部署的 OceanBase 集群
2. 实现更直观的运维操作，支持不在同一个资源定义中的实例运维

更 open

1. 和更多的工具结合，ob-operator 提供了基础的运维能力，结合更多的自动化策略，能实现 OceanBase 的自动化运维
2. OceanBase 作为基础设置，可以和用户的应用集成在一起输出



Thanks.



项目地址: <https://github.com/oceanbase/ob-operator>